

3. This question involves pairing competitors in a tournament into one-on-one matches for one round of the tournament. For example, in a chess tournament, the competitors are the individual chess players. A game of chess involving two players is a match. The winner of each match goes on to a match in the next round of the tournament. Since half of the players are eliminated in each round of the tournament, there is eventually a final round consisting of one match and two competitors. The winner of that match is considered the winner of the tournament.

Competitors, matches, and rounds of the tournament are represented by the `Competitor`, `Match`, and `Round` classes. You will write the constructor and one method of the `Round` class.

```
/** A single competitor in the tournament */
public class Competitor
{
    /** The competitor's name and rank */
    private String name;
    private int rank;

    /**
     * Assigns n to name and initialRank to rank
     * Precondition: initialRank >= 1
     */
    public Competitor(String n, int initialRank)
    { /* implementation not shown */ }

    /* There may be instance variables, constructors,
       and methods that are not shown. */
}
```

```
/** A match between two competitors */
public class Match
{
    public Match(Competitor one, Competitor two)
    { /* implementation not shown */ }

    /* There may be instance variables, constructors,
       and methods that are not shown. */
}

/** A single round of the tournament */
public class Round
{
    /** The list of competitors participating in this round */
    private ArrayList<Competitor> competitorList;

    /** Initializes competitorList, as described in part (a) */
    public Round(String[] names)
    { /* to be implemented in part (a) */ }

    /**
     * Creates an ArrayList of Match objects for the next round
     * of the tournament, as described in part (b)
     * Preconditions: competitorList contains at least one element.
     *                  competitorList is ordered from best to worst rank.
     * Postcondition: competitorList is unchanged.
     */
    public ArrayList<Match> buildMatches()
    { /* to be implemented in part (b) */ }

    /* There may be instance variables, constructors,
       and methods that are not shown. */
}
```

- A. Write the constructor for the `Round` class. The constructor should initialize `competitorList` to contain one `Competitor` object for each name in the `String[]` `names`. The order in which `Competitor` objects appear in `competitorList` should be the same as the order in which they appear in `names`, and the rank of each competitor is based on the competitor's position in `names`. Names are listed in `names` in order from the best-ranked competitor with rank 1 to the worst-ranked competitor with rank n , where n is the number of elements in `names`.

For example, assume the following code segment is executed.

```
String[] players = {"Alex", "Ben", "Cara"};
Round r = new Round(players);
```

The following shows the contents of `competitorList` in `r` after the constructor has finished executing.

0	1	2
name: "Alex" rank: 1	name: "Ben" rank: 2	name: "Cara" rank: 3

Complete the `Round` constructor.

```
/** Initializes competitorList, as described in part (a) */
public Round(String[] names)
```

B. Write the `Round` method `buildMatches`. This method should return a new `ArrayList<Match>` object by pairing competitors from `competitorList` according to the following rules.

- If the number of competitors in `competitorList` is even, the best-ranked competitor is paired with the worst-ranked competitor, the second-best-ranked competitor is paired with the second-worst-ranked competitor, etc.
- If the number of competitors in `competitorList` is odd, the competitor with the best rank is ignored and the remaining competitors are paired according to the rule for an even number of competitors.

Each pair of competitors is used to create a `Match` object that should be added to the `ArrayList` to return. Competitors may appear in either order in a `Match` object, and matches may appear in any order in the returned `ArrayList`.

The following example shows the contents of `competitorList` in a `Round` object `r1` containing an odd number of competitors and the `ArrayList` of `Match` objects that should be returned by the call `r1.buildMatches()`.

`competitorList`:

0	1	2
name: "Alex" rank: 1	name: "Ben" rank: 2	name: "Cara" rank: 3

The `ArrayList` to be returned contains a single match between Ben and Cara, the second and third-ranked competitors:

0
First Competitor: name: "Ben" rank: 2 Second Competitor: name: "Cara" rank: 3

The next example shows the contents of `competitorList` in a `Round` object `r2` containing an even number of competitors and the `ArrayList` of `Match` objects that should be returned by the call `r2.buildMatches()`.

`competitorList`:

0	1	2	3
name: "Rei" rank: 1	name: "Sam" rank: 2	name: "Vi" rank: 3	name: "Tim" rank: 4

The `ArrayList` to be returned contains two matches: one match between the first- and last-ranked competitors Rei and Tim, and a second match between the second- and third-ranked competitors Sam and Vi:

0	1
First Competitor: name: "Rei" rank: 1 Second Competitor: name: "Tim" rank: 4	First Competitor: name: "Sam" rank: 2 Second Competitor: name: "Vi" rank: 3

Complete the `buildMatches` method.

```
/**
 * Creates an ArrayList of Match objects for the next round
 * of the tournament, as described in part (b)
 * Preconditions: competitorList contains at least one element.
 *                competitorList is ordered from best to worst rank.
 * Postcondition: competitorList is unchanged.
 */
public ArrayList<Match> buildMatches()
```